Machine Learning Progress Report

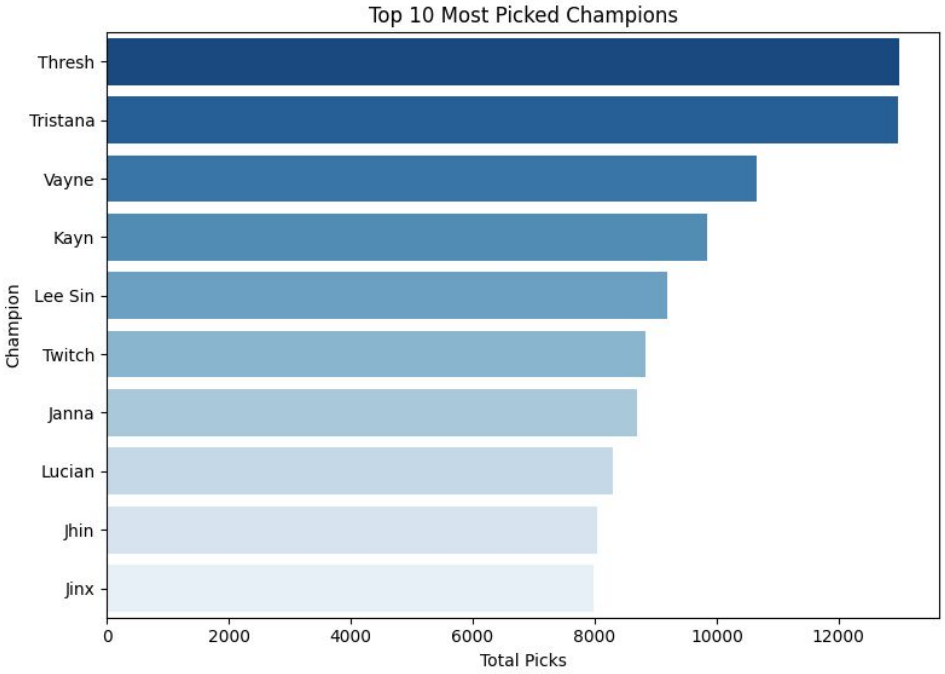
Barrett Clark, Michael Garza, Musa Blake

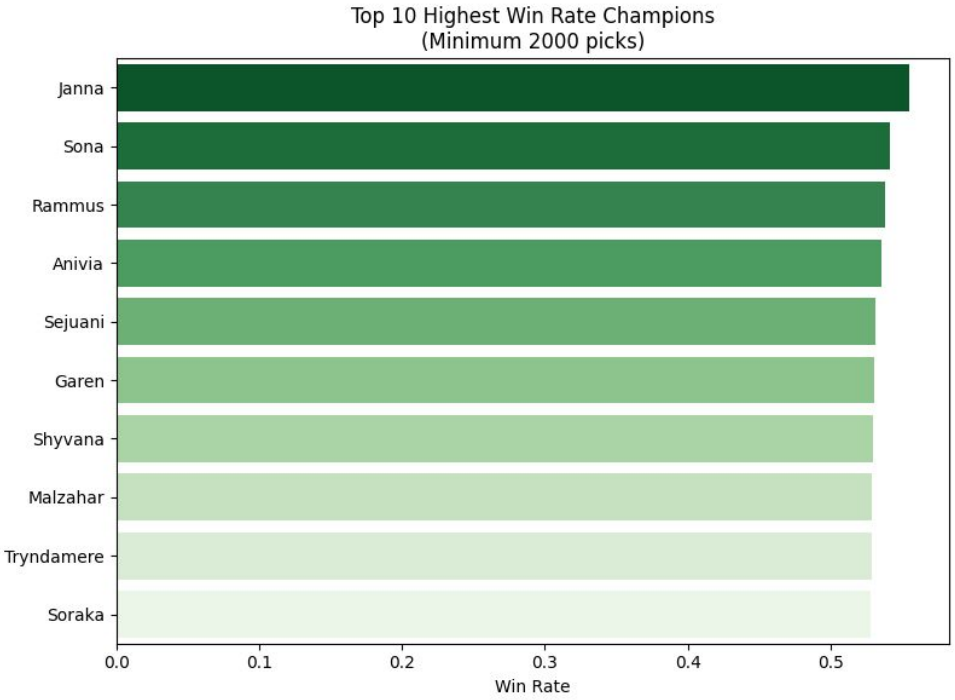
Preliminary Testing and EDA

- Started with looking through the data
- Check champion win rates
- Champion popularity
- Encoded champion picks as features

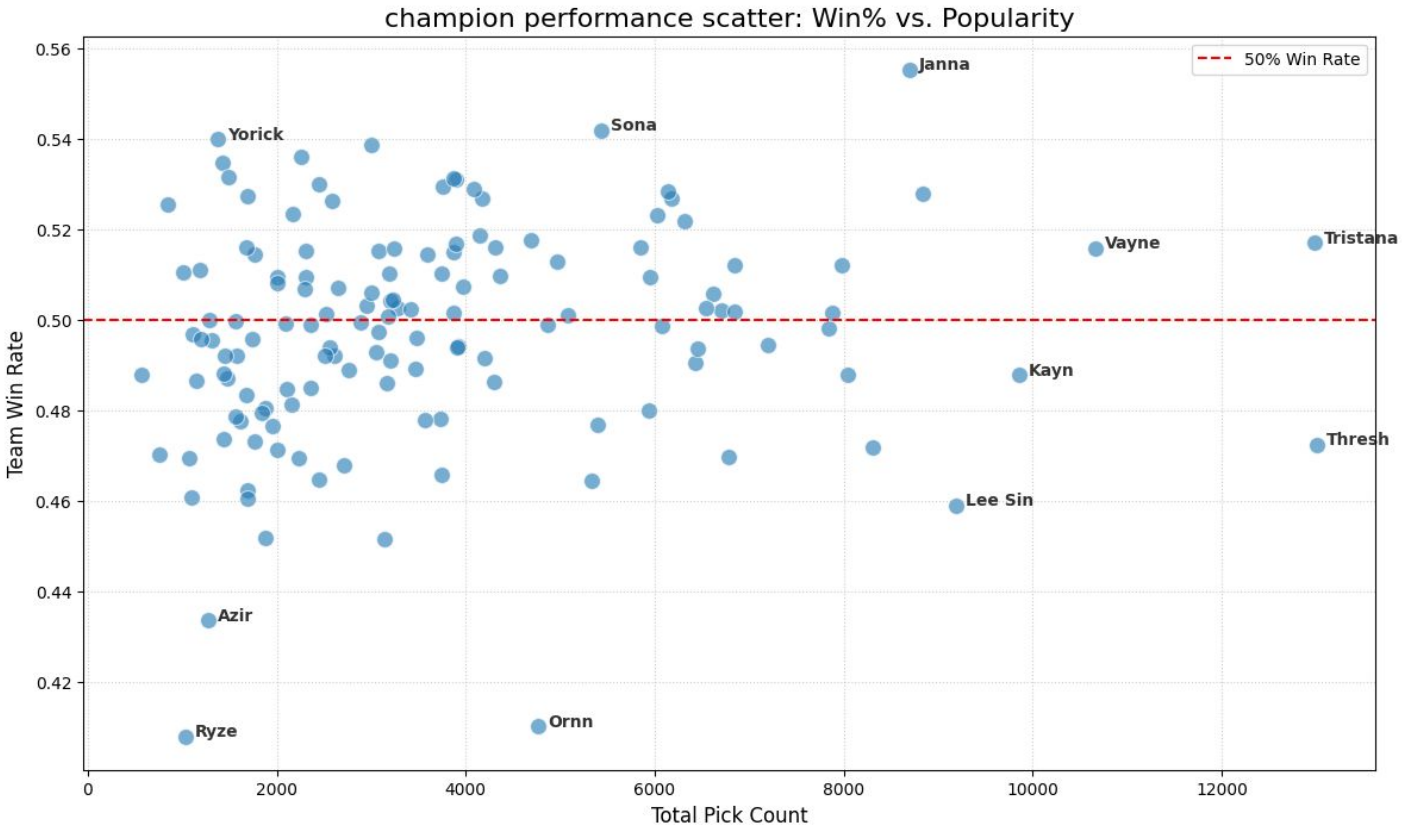
Initial EDA and Dataset Testing

Popular Champions and Highest Win Rates





All Champion Win Rates Scatter Plot



One Hot Encoding

```python
t2_df = df_games[['winner'] + t2_cols].copy()
t2_df['win'] = (t2_df['winner'] == 2).astype(int)
t2_melt = t2_df.melt(id_vars=['win'], value_vars=t2_cols, value_name='champ_id')

# combine
df_picks = pd.concat([t1_melt[['champ_id', 'win']], t2_melt[['champ_id', 'win']]])
df_picks['champ_name'] = df_picks['champ_id'].map(champ_dict)

# win-rate/pick-count calc
champ_stats = df_picks.groupby('champ_name').agg(
    picks=('win', 'count'),
    wins=('win', 'sum')
).reset_index()

champ_stats['win_rate'] = champ_stats['wins'] / champ_stats['picks']

display(champ_stats.sort_values('picks', ascending=False))
```

	champ_name	picks	wins	win_rate
110	Thresh	13002	6143	0.472466
111	Tristana	12983	6713	0.517061
119	Vayne	10658	5498	0.515857
53	Kayn	9853	4807	0.487872
60	Lee Sin	9188	4217	0.458968
...	...	...	...	...
92	Ryze	1032	421	0.407946
49	Karthus	1007	514	0.510427
8	Aurelion Sol	841	442	0.525565
0	Aatrox	753	354	0.470120
100	Skarner	574	280	0.487805

138 rows × 4 columns

```python
        if champ_id in game_dict:
            game_dict[champ_id] = 1

    for i in range(1, 6):                          # Team 2's 5 picks set to -1
        champ_id = row[f't2_champ{i}id']
        if champ_id in game_dict:
            game_dict[champ_id] = -1

    game_dict['winner'] = row['winner']            # attach the label (1 or 2)
    game_features.append(game_dict)


# create the new dataframe with our champion presence features and winner
df_champion_presence = pd.DataFrame(game_features)

print("Shape:", df_champion_presence.shape)
display(df_champion_presence)
```

Shape: (51490, 139)

	1	2	3	4	5	6	7	8	9	10	...	412	420	421	427	429	432	497	498	516	winner
0	0	0	0	0	0	0	0	1	0	0	...	-1	0	0	0	0	1	0	-1	0	1
1	0	0	0	0	0	0	0	0	0	1	...	0	0	0	0	0	0	0	0	0	1
2	0	0	0	0	0	0	0	0	0	0	...	-1	0	0	0	0	0	0	0	0	1
3	0	0	0	0	0	0	0	0	0	0	...	-1	0	0	0	0	0	0	0	0	1
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
51485	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	1	1	0	0	0	2
51486	0	0	0	0	0	0	0	0	0	0	...	1	0	0	0	-1	0	0	0	0	2
51487	-1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	2
51488	0	0	0	0	0	0	0	0	0	0	...	0	1	0	0	0	0	0	0	0	2
51489	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	1

51490 rows × 139 columns

Exploring New Datasets

- This data set gave us good test score results
- We didn’t like the fact that this was old data
- A player today would couldn’t actually use this
- 7.17.2 data - Before I even started playing
- 32 champions are not included in this data set
- We wanted to continue data hunting

Dataset Hunting

CALIFORNIANBILL · UPDATED 8 MONTHS AGO

23 Code Download

Patch 25.14+ (LoL) League of Legends Ranked Games

100k+ Games full match data collected from Riot's API

Usability
10.00

License
CC0: Public Domain

Expected update frequency
Never

Tags
Data Analytics
Games
Video Games
Statistical Analysis
Recommender Systems

(LoL) League of Legends Ranked Games

Details from over 50,000 ranked games of LoL

Data Card Code (26) Discussion (12) Suggestions (0)

eague of Legends, as well as json files containing a way to convert
game, there are fields for:

m1, 2 = team2, 0 = none)
champion and summoner spell IDs)
each team has

sy to lookup and collect information on a users ranked history and
t, in this case I am using a list of usernames scraped from 3rd party LoL

Usability
7.65

License
CC0: Public Domain

Expected update frequency
Not specified

Tags
Internet
Video Games

NATHAN SMALLCALDER · UPDATED 3 MONTHS AGO

31 Code Download

League of Legends(LoL) Matches Patch 25.19+

Comprehensive Match Stats from 270,000+ League Matches

ena (MOBA)
statistics,

ctured data

Usability
10.00

License
Other (specified in description)

Expected update frequency
Weekly

Tags
Tabular
Data Analytics
Beginner
Video Games
Intermediate

Datasets: gptilt / lol-ultimate-events-challenger-10m

like 2 Follow GPTilt - Open-Source L... 1

Modalities: Tabular Text Formats: parquet Size: 10M - 100M Libraries: Datasets Dask Croissant +1

Dataset card Data Studio Files and versions xet Community 1

main lol-ultimate-events-challenger-10m / README.md

franciscoabsampaio Add dataset card 61250f9 VERIFIED 11 months ago

Preview Code raw Copy download link history blame contribute delete 10.7 kB

metadata

```
configs:
  - config_name: events
    data_files:
      - split: train_region_americas
        path: events/train_region_americas*.parquet
      - split: test_region_americas
        path: events/test_region_americas*.parquet
      - split: train_region_asia
        path: events/train_region_asia*.parquet
      - split: test_region_asia
        path: events/test_region_asia*.parquet
      - split: train_region_europe
        path: events/train_region_europe*.parquet
      - split: test_region_europe
        path: events/test_region_europe*.parquet
```

GPTilt: 10M Enriched Events from 10K LoL Challenger Matches

This dataset is part of the GPTilt open-source initiative, aimed at democratizing access to high-quality LoL data for research and analysis, fostering public exploration, and advancing the community's understanding of League of Legends through data science and AI. It provides detailed data from high-elo matches.

By using this dataset, users accept full responsibility for any consequences arising from its use. GPTilt assumes no liability for any damages that may result. Users are strongly encouraged to review the "Uses" section—particularly the "Out-of-Scope Use" subsection—for guidance.

Dataset Comparison Benchmarks

```
datasnaek          0.5469 +/- 0.0050  folds: [np.float64(0.5423), np.float64(0.5421), np.float64(0.5457), np.float64(0.5556), np.float64(0.5486)]
californianbill    0.5186 +/- 0.0036  folds: [np.float64(0.5173), np.float64(0.5122), np.float64(0.5217), np.float64(0.5197), np.float64(0.5219)]
gptilt             0.4182 +/- 0.0618  folds: [np.float64(0.4242), np.float64(0.303), np.float64(0.4848), np.float64(0.4242), np.float64(0.4545)]
nathansmallcalder  0.5215 +/- 0.0042  folds: [np.float64(0.5295), np.float64(0.5217), np.float64(0.5181), np.float64(0.5199), np.float64(0.5184)]
```

- Initial Tests Proved to not have enough randomization/folds
- While datasnaek has the highest test score, it was based on older, more unbalanced patches. There was less champion variety and games were more predictable
- We decided we needed more thorough testing

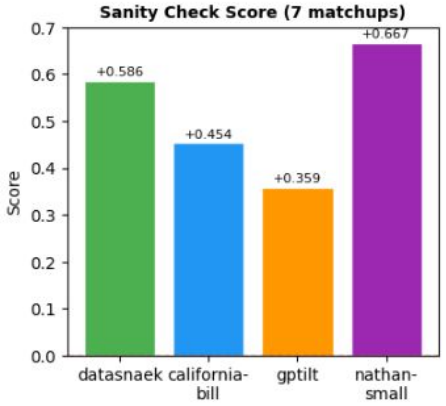
Dataset Comparison Benchmarks

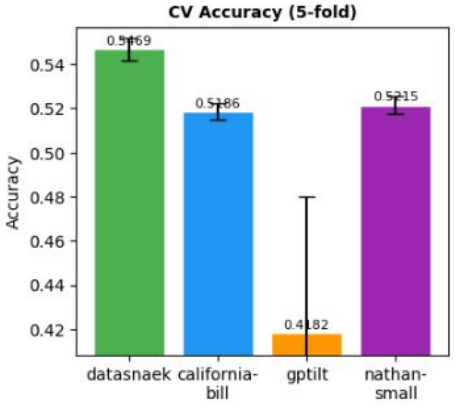
New Combination Metrics

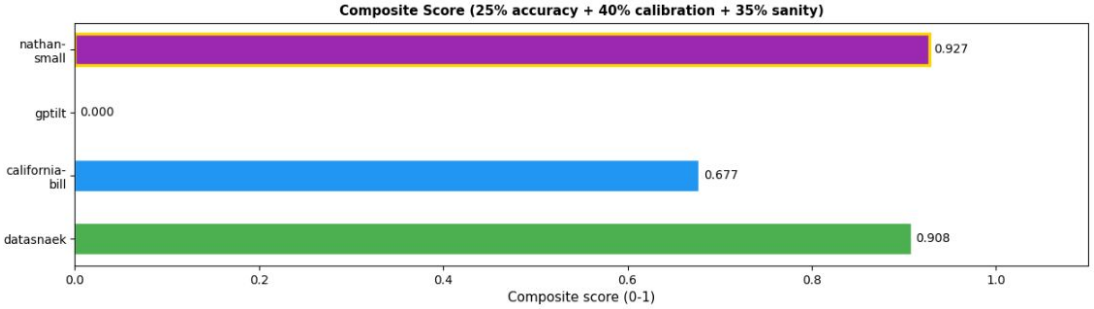
- Test Scores
  - Accuracy Simply Checks if you got the winner right (99% prediction vs 51% still got it right)
- Calibration Scores
  - Calibration checks if our percentage predictions are correct (the closer the percentages matter more)
- Obvious Sanity Checks

Dataset Comparison Benchmarks

```
Sanity check scores (positive = correct direction)

Full AD into Malphite and tank comp (expect: t2)
  [MARGINAL] datasnaek              T1=49.9%  score=+0.001
  [MARGINAL] californianbill        T1=49.6%  score=+0.004
  [FAIL] gptilt                 T1=59.0%  score=-0.090
  [PASS] nathansmallcalder      T1=44.7%  score=+0.053

Squishy assassins vs full tank frontline (expect: t2)
  [MARGINAL] datasnaek              T1=51.2%  score=-0.012
  [MARGINAL] californianbill        T1=48.8%  score=+0.012
  [FAIL] gptilt                 T1=53.6%  score=-0.036
  [PASS] nathansmallcalder      T1=41.6%  score=+0.084

High win-rate champs vs low win-rate champs (expect: t1)
  [FAIL] datasnaek              T1=47.6%  score=-0.024
  [MARGINAL] californianbill        T1=49.3%  score=-0.007
  [PASS] gptilt                 T1=57.3%  score=+0.073
  [PASS] nathansmallcalder      T1=53.1%  score=+0.031

Mirror match (expect: even)
  [PASS] datasnaek              T1=49.1%  score=+0.482
  [PASS] californianbill        T1=52.1%  score=+0.458
  [PASS] gptilt                 T1=42.8%  score=+0.357
  [PASS] nathansmallcalder      T1=49.1%  score=+0.481

Poke comp vs dive comp (expect: t2)
  [PASS] datasnaek              T1=45.4%  score=+0.046
  [MARGINAL] californianbill        T1=48.5%  score=+0.015
  [FAIL] gptilt                 T1=52.1%  score=-0.021
  [PASS] nathansmallcalder      T1=48.0%  score=+0.020

Protect-the-ADC vs all-in dive (expect: t1)
  [PASS] datasnaek              T1=55.1%  score=+0.051
  [FAIL] californianbill        T1=47.1%  score=-0.029
  [PASS] gptilt                 T1=52.2%  score=+0.022
  [MARGINAL] nathansmallcalder      T1=48.3%  score=-0.017

Historically strong vs historically weak champs (expect: t1)
  [PASS] datasnaek              T1=54.1%  score=+0.041
  [MARGINAL] californianbill        T1=50.1%  score=+0.001
  [PASS] gptilt                 T1=55.4%  score=+0.054
  [MARGINAL] nathansmallcalder      T1=51.4%  score=+0.014

Total sanity scores (max = 3.5 per dataset)
  datasnaek             total=+0.586  passed=4/7
  californianbill       total=+0.454  passed=1/7
  gptilt                total=+0.359  passed=4/7
  nathansmallcalder     total=+0.667  passed=5/7
```







Final Choice

- We valued a newer, complete dataset rather than more data.
- Nathansmallcalder had the best composite score, and the most complete dataset (including all champions)

NATHAN SMALLCALDER · UPDATED 3 MONTHS AGO

League of Legends(LoL) Matches Patch 25.19+

Comprehensive Match Stats from 270,000+ League Matches

Data Card Code (3) Discussion (1) Suggestions (0)

Rerun with new data

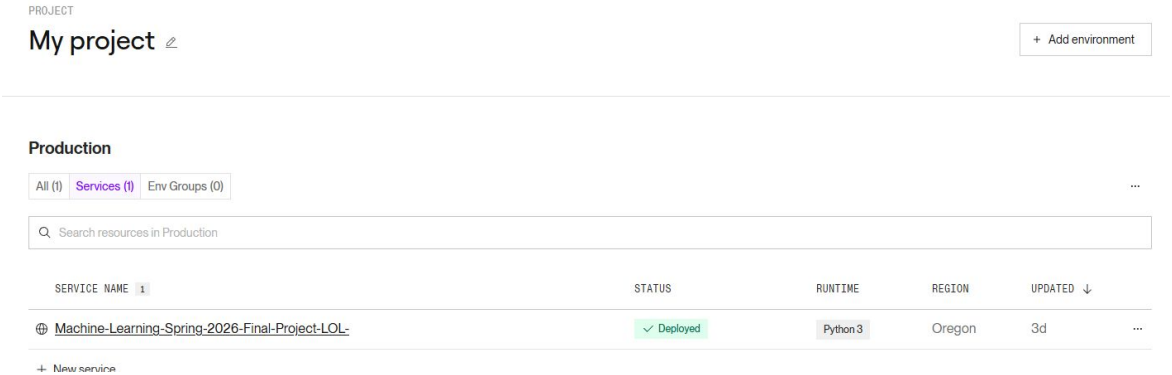
- Filtered the data to classic games
- Transformed champions into features
- Trained the best classifier (random forest)
- After training the classifier, we can plug in our features (all 10 champions), and predict the winrate result
- In order to find the “next optimal champion”, we simply plug in all features before the next pick, then iterate through checking every next pick, and give back the rank the targets by highest to lowest win rate
- In order to limit these by roles, we create an array of what champions correspond to roles and limit the options to that

```
Best JUNGLE picks for BLUE team:
  Blue: ['Jinx', 'Thresh', 'Ahri']
  Red:  ['Malphite', 'Lux']

  Champion              Win Prob
  ------------------------------
  Lillia                  52.0%
  Ivern                   52.0%
  Fiddlesticks            51.1%
  Diana                   49.7%
  Zac                     49.2%
```

[Classifier Training w Proper Data]

Implementation

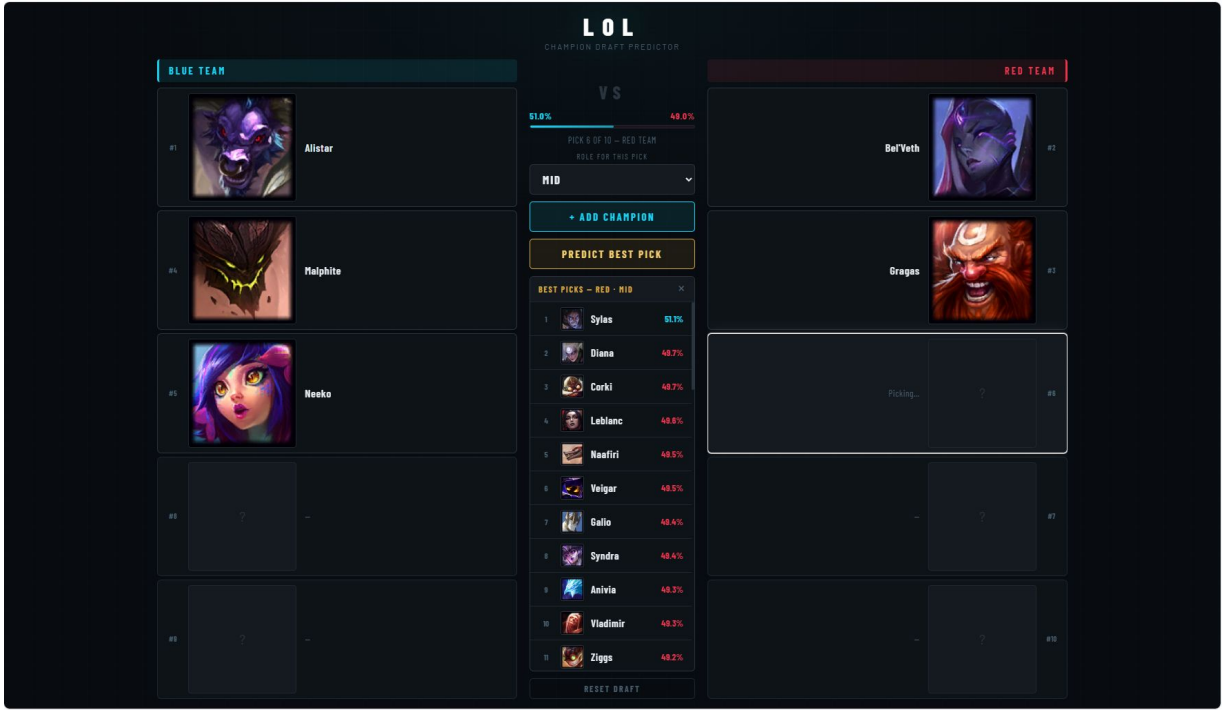
- Github Hosts a free webpage, but classifier was too large
- We exported our trained model as a pkl file
- We instead made a React Account to Host an API
- Vibe coded a UI to allow an interactive champion drafting experience with our classifier



Predictor In Action

Full AD VS Malphite = + 6.4%

Predictor In Action



Best Ults = Sylas

- Winrates Reference
- Live Demo!

Clustering

We have 149,717 Classic matches in the kaggle dataset. The question we care about is: do certain 5-champion team compositions win more often than others? This verifies our tool is useful and is finding real patterns, but it's a hard question to answer by looking at just the data because there are millions of possible 5-champion combinations.

Cluster analysis helps by grouping compositions that are structurally similar.

Clustering Methodology

1. **Filtered to Classic mode only.**

   ARAM and URF have completely different rules and meta. Including them would muddy the signal. Classic accounts for 190,788 of 279,423 total matches.

2. **One-hot encoded each team's 5 picks.**

   Every team becomes a vector (one binary feature per champion in the pool). A 1 means that champion was picked; 0 means they weren't. Every team's vector has exactly five 1s. This is the same format our random forest uses as input.

3. **Reshaped to one row per team (not per match).**

   The original data had one row per match with both teams. We split it so each row is one team's picks plus a win/loss label. This doubles our rows, and analyzes both team comps in a given match.

4. **Ran UMAP to compress 172 dimensions down to 2.**

   Obviously can't visualize or cluster 172-dimensional data directly. UMAP projects it into a 2D map where similar compositions land close together.

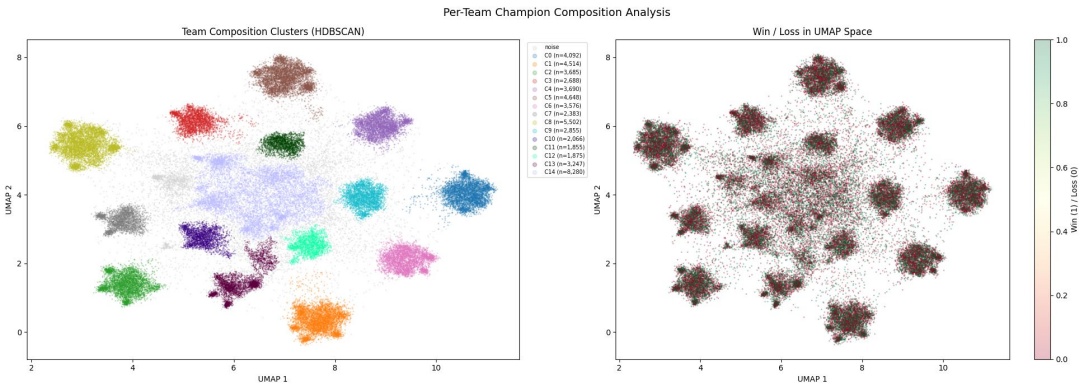
5. **Ran HDBSCAN to find clusters in that 2D map.**

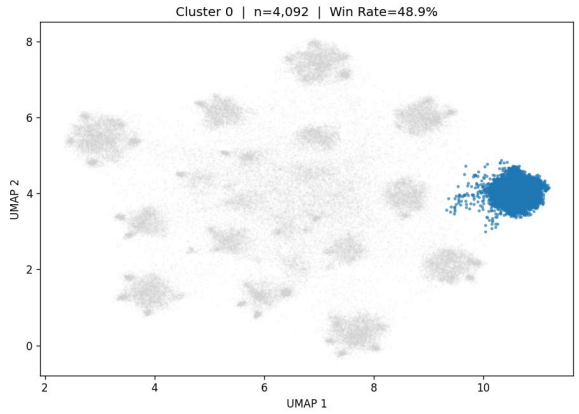
   HDBSCAN automatically identifies the dense areas in the UMAP plot and labels them as composition "archetypes".

6. **Analyzed each cluster:**

   Win rate and the most over-represented champion per role* (top, jungle, mid, ADC, support) to describe what each archetype might sorta look like.

Clustering





C	n	WR	95% CI	>=1 Def%	TOP	JG	MID	ADC	SUP
11	1855	51.4%	[49.2%, 53.7%]	98.0%	Gwen (top)	XinZhao (jun)	Sylas (mid)	Sivir (adc)	Yuumi (sup)
10	2066	51.4%	[49.2%, 53.6%]	96.0%	Chogath (top)	Shyvana (jun)	Qiyana (mid)	Tristana (adc)	Lulu (sup)
2	3685	51.3%	[49.7%, 52.9%]	98.6%	Jax (top)	Volibear (jun)	Diana (mid)	Jinx (adc)	Lulu (sup)
1	4514	51.1%	[49.6%, 52.6%]	97.4%	Sett (top)	Amumu (jun)	Diana (mid)	MissFortune (adc)	Leona (sup)
13	3247	50.7%	[49.0%, 52.4%]	96.2%	Mordekaiser (top)	Karthus (jun)	Naafiri (mid)	Lucian (adc)	Nami (sup)
12	1875	50.7%	[48.5%, 53.0%]	95.7%	Mordekaiser (top)	Briar (jun)	Anivia (mid)	Vayne (adc)	Lulu (sup)
9	2855	50.0%	[48.2%, 51.9%]	97.5%	Malphite (top)	Zac (jun)	Malzahar (mid)	Smolder (adc)	Nami (sup)
6	3576	49.7%	[48.1%, 51.4%]	98.9%	Ornn (top)	MasterYi (jun)	Diana (mid)	Ashe (adc)	Seraphine (sup)
14	8280	49.3%	[48.2%, 50.4%]	30.9%	Aatrox (top)	Karthus (jun)	Yone (mid)	Draven (adc)	Senna (sup)
8	5502	49.1%	[47.8%, 50.4%]	99.2%	Renekton (top)	LeeSin (jun)	Viego (mid)	Kaisa (adc)	Nautilus (sup)
0	4092	48.9%	[47.3%, 50.4%]	98.1%	Illaoi (top)	Warwick (jun)	Akali (mid)	Jhin (adc)	Xerath (sup)
7	2383	48.2%	[46.2%, 50.2%]	99.0%	Gwen (top)	LeeSin (jun)	Sylas (mid)	Aphelios (adc)	Thresh (sup)
5	4648	47.9%	[46.5%, 49.4%]	98.8%	Teemo (top)	Kayn (jun)	Ekko (mid)	Caitlyn (adc)	Lux (sup)
4	3690	47.9%	[46.3%, 49.5%]	98.8%	Ornn (top)	LeeSin (jun)	Zed (mid)	Ezreal (adc)	Karma (sup)
3	2688	47.4%	[45.6%, 49.3%]	95.9%	Sett (top)	LeeSin (jun)	Ahri (mid)	Yunara (adc)	Lulu (sup)

fun plot w/ plotly

Level of Importance

“The following code is a standard approach to analyzing who wins a League of Legends match and why.”

```
import pandas as pd

importance_df = pd.DataFrame({
    "feature": X.columns,
    "importance": result.importances_mean
}).sort_values(by="importance", ascending=False)

print(importance_df)
```

Target variable BlueWin(did blue side win?)

Features : Tower Kills, Champion Picks, Dragon/Baron/Rift Herald kills, and Team kills.

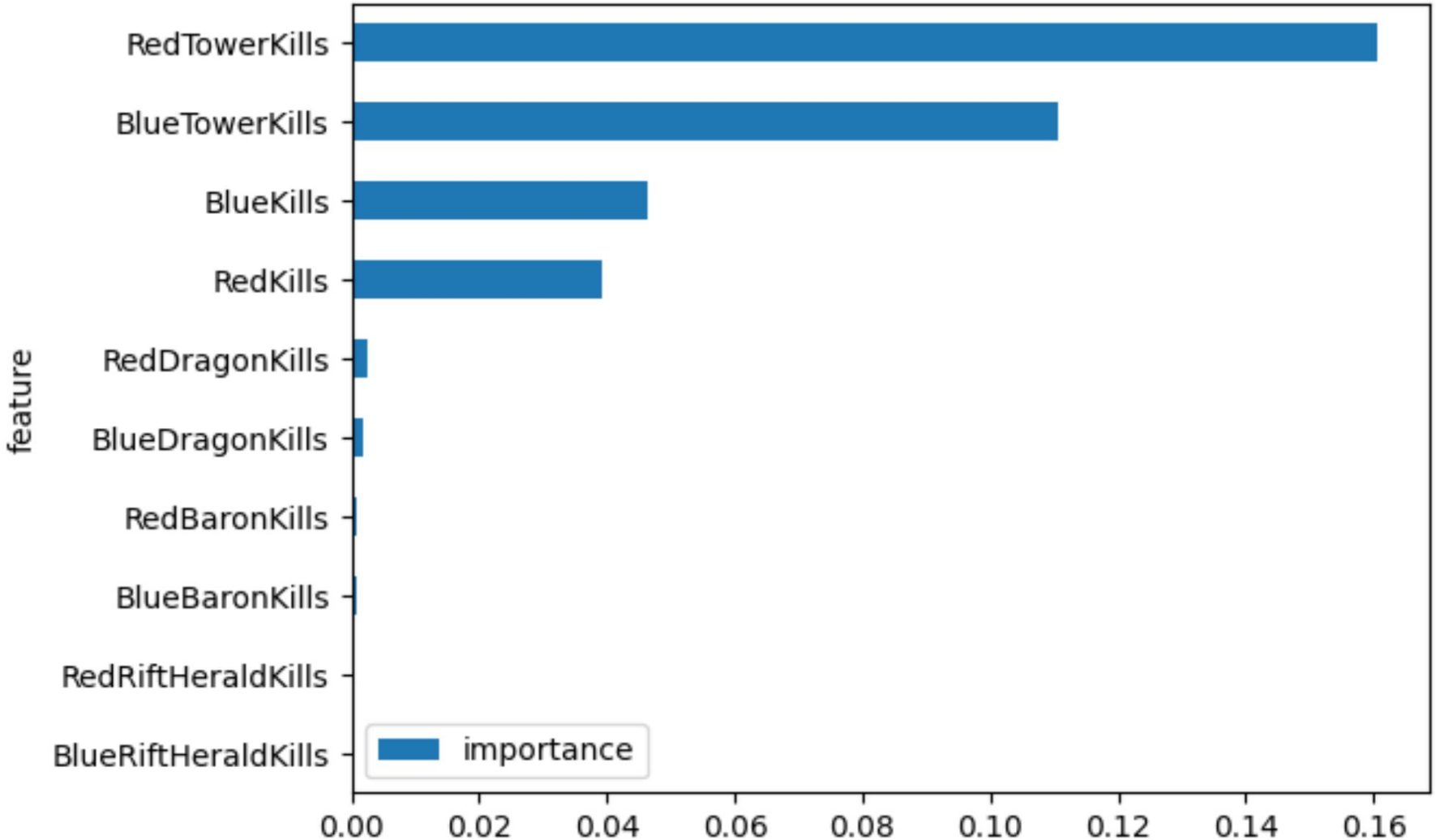
Model: Random Forest Classifier

80/20 train/test split

I choose permutual importance because it measured real predictive impact rather than just how frequently a feature was used during training.

feature	importance
RedTowerKills	0.160535
BlueTowerKills	0.110547
BlueKills	0.046370
RedKills	0.039166
RedDragonKills	0.002578
BlueDragonKills	0.001964
RedBaronKills	0.001003
BlueBaronKills	0.000742
RedRiftHeraldKills	0.000597
BlueRiftHeraldKills	0.000357
B1Champ	0.000278
B2Champ	0.000166
R5Champ	0.000035
R2Champ	-0.000009
B4Champ	-0.000037
R1Champ	-0.000040
R4Champ	-0.000051
B5Champ	-0.000100
R3Champ	-0.000103
B3Champ	-0.000203
TeamID	-0.000338

In order from the most important to the least important to win the battle. Having a negative is basically a 0%.


RedTowerKills
BlueTowerKills
BlueKills
RedKills
RedDragonKills
BlueDragonKills
RedBaronKills
BlueBaronKills
RedRiftHeraldKills
BlueRiftHeraldKills
feature
importance
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.16

Key Findings

-Tower Kills are the number #1 win condition (more impactful that kills)

-Kills matter but less than towers

-Towers are 3-4 times more important

-Champions selection hardly matter here.

- Dragon/Baron/Herald have minimal impact

Next Steps

- Duplicate dataset and encode everything opposite. See if that helps scoring.
- Add “details” option -> poster board explanation of our work for future users.
- Possibly accumulate all of our collab findings into one ipynb (we have 8?)
- Unsupervised Learning?
- Check if our dataset has ranked and explore that functionality
- Better understand each others data exploration